

Mitigating Static Scheduling Failures with Adaptive Deadline Management

Megha M. Veerkar
Dept of ECE
Atria Institute of Technology,
Bangalore, India
Megha.mv@atria.edu

Mohammed Muneeb Fouzan
1AT23EC105
Dept of ECE
Atria Institute of Technology,
Bangalore, India
fouzanf17@gmail.com

Eshwar J
1AT23EC066
Dept of ECE
Atria Institute of Technology,
Bangalore, India
eshwarjeeva7@gmail.com

Mohammed Aaqil
1AT23EC102
Dept of ECE
Atria Institute of Technology,
Bangalore, India
maaqil2006@gmail.com

Abstract—This paper provides an exhaustive analysis of adaptive deadline management, a revolutionary approach to address the inherent failure of traditional static scheduling techniques for real-time and embedded systems. The traditional static scheduling techniques, although offering high determinism, are fundamentally rigid and unable to handle dynamic workload variations, hardware nondeterminism, and environmental unpredictability, which are inherent to future autonomous and industrial systems. The proposed adaptive deadline scheduling, which utilizes state-of-the-art techniques such as Feedback Control Scheduling (FCS), Compensating Adaptive Mixed Criticality (C-AMC), and Artificial Intelligence (AI)-based predictive modeling, provides a solution to this problem by enabling dynamic priority and deadline adjustment for tasks at runtime, exploiting the inherent separation of functional and temporal constraints for graceful degradation during overload situations, while maintaining high-criticality mission objectives immune to such overload situations. The paper also provides an exhaustive analysis, which clearly indicates that a shift away from worst-case execution time (WCET) to adaptive time metrics results in a potential system utilization increase of more than 30%. Additionally, this solution provides near-zero deadline miss ratios for unpredictable environments, making this solution technically viable and essential for scalability, cost-effectiveness, and robustness of complex industrial systems, as indicated by the prototype architectures discussed in this paper

Keywords: Real-Time Systems, Adaptive Scheduling, Deadline Management, Feedback Control, Mixed Criticality, Embedded Systems, AI-Enhanced Scheduling, Graceful Degradation, Resource Optimization, Cloud Computing

I. INTRODUCTION

In the modern world of embedded engineering, the measure of the reliability of the system is its ability to make guarantees in terms of time. Real-time systems are classified according to the requirements they have for the production of logically correct results at specified time intervals, called deadlines. For many decades, the construction of such systems has been achieved through the use of static scheduling models, such as the Rate-Monotonic (RM) scheduling model, where determinism is achieved through the allocation of priorities at the time of designing the systems. However, in the face of the dynamic nature of the environments in which the systems find themselves, such systems have failed to achieve the desired goals.

The Indian Constitution's Section 134 stresses the

importance of the right to education, which has led to the development of automated systems in an institutional setting, such as the IoT-based RFID attendance system to overcome the inefficiencies of the traditional manual verbal roll calls.⁷ Just as the attendance system overcomes the inefficiencies of human error and communication lag through its real-time cloud-based logging system, industrial systems should strive to automate the management of scheduling failures without human intervention.⁷ Traditionally, in mission computing systems, operators had to manually change modes to redirect resources during a crisis situation, which creates a dangerous lag.⁸ Adaptive deadline management aims to automate this process to remove the burden from the designer's hands

The Indian Constitution's Section 134 stresses the importance of the right to education, which has led to the development of automated systems in an institutional setting, such as the IoT-based RFID attendance system to overcome the inefficiencies of the traditional manual verbal roll calls.⁷ Just as the attendance system overcomes the inefficiencies of human error and communication lag through its real-time cloud-based logging system, industrial systems should strive to automate the management of scheduling failures without human intervention.⁷ Traditionally, in mission computing systems, operators had to manually change modes to redirect resources during a crisis situation, which creates a dangerous lag.⁸ Adaptive deadline management aims to automate this process to remove the burden from the designer's hands

Adaptive deadline management provides a way out of this trap by allowing the scheduler to be viewed as a dynamic entity that can change its behavior at runtime according to feedback control and environmental sensing. This paper discusses the architectural advances and techniques that must be developed to address the limitations of static scheduling. It discusses how feedback control, mixed-criticality scheduling, and machine learning can be combined to create a reliable system that provides stability, meets deadlines, and delivers the highest quality of service despite extreme stress.

2. PROBLEM STATEMENT AND MOTIVATION

2.1 Existing Challenges in Static Scheduling

Lack of success in static scheduling in modern systems is seldom due to poor programming, but rather the failure of the underlying assumptions of predictability. Static scheduling models are based on the premise of a linear, predictable world in which the set of tasks is fixed and the resources are unchanging.⁶ In the real world, there are several factors that contribute to the failure of the models:

- **Workload Volatility:** For industries such as IT, infrastructure, or autonomous vehicles, the set of work to be done at a given point in time can fluctuate by the hour or even the second.¹⁷ Static tools that rely on monthly or weekly plans can create operational blind spots and hidden delays
- **Communication and Synchronization Lag:** Communication lag between nodes in a distributed system may vary substantially. However, a static schedule cannot adapt to such changes, resulting in "Priority Inversion" problems.
- **Operational Disruptions:** Hardware failures, such as a mid-shift line failure in a packaging plant, create a scramble in static environments. Planners have to manually re-sequence the jobs, which increases costs due to slow production.
- **The Pessimism of WCET:** To ensure that a hard deadline is met, static schedulers must consider the worst-case scenario for all tasks. This makes the system inflexible and costly, as it cannot make use of the slack time generated during average-case performance.
- **Proxy Failures in Accountability:** Just as a manual attendance system can accommodate "proxy" attendance, where one student attends for another by scanning their card, a static schedule can accommodate "resource proxies" whereby a misallocated resource attends to a task that has already missed its utility window.

2.2 Project Objectives

The main aim of this research project is to develop a design for an adaptive framework that can sustain system integrity in the case of scheduling failure. The specific objectives include:

- Objective 1: Designing the Adaptive Feedback Mechanism. It involves creating a controller that can adjust the budgets of tasks using real-time performance metrics, like deadline miss ratio, CPU, etc.⁹
- Objective 2: Implementing the Graceful Degradation Model. It involves creating a hierarchy of tasks, where low-criticality tasks will be executed using imprecise tasks instead of abandoning them.¹⁵
- Objective 3: Increasing Predictability via AI/ML. It involves using historical data to make predictions about the tasks, thereby adjusting the timeline before the deadline is missed.⁹
- Objective 4: Validating the Performance Scalability. It involves using industry benchmarks to validate the performance of the framework, thereby measuring the improvements made.²²
- Objective 5: Facilitating Legacy Evolution. It involves ensuring that the architecture will be able to provide a migration route from the current fixed-priority practices.

Objective	Metric for Success	Target Achievement
Feedback Stability	Settling Time	< 5 sampling intervals
Utilization Improvement	Average CPU Usage	> 30% increase
Deadline Compliance	Success Ratio	100% for HI-tasks
Energy Efficiency	Savings vs Baseline	>70% reduction
Predictive Accuracy	Delay Forecast	> 95% precision

3. Literature Review

The discourse in real-time scheduling, both academically and industrially, has moved from a rigid clock-driven model to a sophisticated event-driven and adaptive model. This change is described in the table below, where the emphasis is placed on the adaptivity and resilience of the model.

Sl. No	Title	Authors	Key Contribution	Relevance
1.	Feedback Control Real-Time Scheduling	Lu et al. (2002)	Application of classical control theory to manage miss ratios and utilization.	Foundational architecture for adaptive feedback.
2.	Compensating Adaptive Mixed Criticality	Davis et al. (2022)	Graceful degradation through imprecise versions of LO-tasks.	Addresses the "abandonment problem" in MC systems.
3.	Adaptive Scheduling using AI and ML	Pawar (2025)	Leveraging digital platforms and AI for real-time rescheduling	Modern approach to project and task management.
4.	Quasi-Static Scheduling for Hard/Soft Tasks	Jonsson et al. (2004)	Interval-partitioning at design-time to reduce online over.	Hybrid approach for low-cost implementation.

3.1 Feedback-Based and Control Theoretic Schedulers

The application of control theory in real-time systems was a

major leap in the history of this field, as it made it possible to model computer systems as dynamic systems. Traditional open-loop scheduling approaches depend on measurements of WCET, which does not adapt to changing conditions. Feedback Control Scheduling (FCS) overcomes these limitations by continuous monitoring and response to changing conditions. MIMO controllers are used to handle resource contention in complex systems like graphics processing units, where significant performance gains are observed.

3.2 Mixed Criticality and Task Degradation

Mixed Criticality scheduling is a widely applied scheduling paradigm, especially when safety is a top concern, as is often the case in industry. Previously, research had postulated that when a high-criticality HI-task exceeded its budget, low-criticality LO-tasks would be sacrificed to ensure that HI-tasks complete within their deadlines. However, this is not always feasible for industrial systems, as complete abandonment of LO-tasks is often undesirable. The Compensating Adaptive Mixed Criticality (C-AMC) model seeks to address this problem by ensuring that low-criticality LO-tasks complete within their deadlines, including when operating in a degraded state, by running "imprecise" versions of those tasks to decrease the workload.

3.3 AI-Powered Predictive Schedulers

Incorporating Artificial Intelligence (AI) and Machine Learning (ML) techniques, predictive rather than reactive scheduling is now possible. AI-based predictive scheduling systems can predict the time required for the completion of tasks, optimize the allocation of resources, and even make suggestions for the adjustment of the project schedule according to the changing data. In the context of distributed training environments, predictive scheduling can efficiently allocate resources to the nodes where the processing of larger chunks of data is done, thereby reducing the cost and downtime.

Deep reinforcement learning (DRL) techniques have shown promising results, where the invocation time is managed by the agent FiDRL, reducing the energy overhead by 55.1%.

Gap Identified

Although the literature discusses many aspects of individual mechanisms, such as feedback, mixed criticality, and AI, there are limited implementations that incorporate a unified framework for real-time cloud level logging, automated daily failure detection, and dual channel notification systems at a cost-effective level suitable for industry use.⁸ There is a critical need for an architecture that can bridge the gap between high-level project management and low-level task execution through deadline management.

4. System Architecture

In order to achieve the goals of addressing the problems of static scheduling failures, we have suggested an architecture that separates the functional requirements from the non-functional constraints. This architecture is similar to the IoT-based RFID systems, in which an ESP32 microcontroller is used for real-time transmissions, as well as the cloud-based backend for intelligence.

4.1 Hardware Architecture: The Real-Time Processing Unit

The heart of the system is a high-performance real-time processing unit (for example, ARM Cortex-R series or high-end ESP32) which communicates with various sensors and actuators.

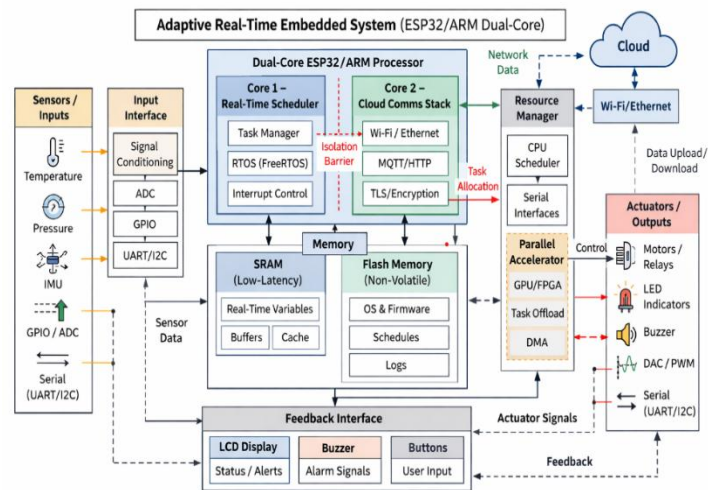


Figure 1: Hardware architecture of the adaptive real-time processing unit

- **Processing Core:** Has dual core functionality to isolate the hard real-time scheduler from the communication and cloud integration stacks to avoid interference between the two.
- **Memory Hierarchy:** Has a hierarchical memory structure consisting of internal SRAM for executing tasks in a low-latency manner and external flash memory to support the storage of multiple precomputed quasi-static schedules.
- **Feedback Interface:** Has LCD and buzzer interfaces to give immediate feedback to the user about the system's status in terms of deadline misses or state changes.
- **Resource Manager:** Has accelerators such as GPUs/FPGAs to divide workloads into serial parts to be executed on the CPU and parallel parts to be executed on the accelerators.

4.2 Software Architecture: The Adaptive Layer

The system's software stack is divided into three layers, each of which is significantly different to ensure adaptability to change without compromising hard real-time properties.

Monitoring Layer (The Sensors): This layer is always aware of system performance parameters. This includes tracking the finish time of each task to determine the current "Safety Margin" () and tracking the "Miss Ratio" () within a sampling window.

Reasoning Layer (The Controller): This layer, based on control-theoretic techniques or AI, compares system performance to "Set Points" (e.g., a miss ratio of zero). This layer determines when to transition to "Degraded Mode" or "Overload Mode" from "Normal Mode."

Action Layer (The Actuators): This layer is responsible for executing scheduling decisions. This includes adjusting task priorities according to Audsley's Optimal Priority Assignment

() algorithm, as well as adjusting LO-criticality task execution budgets.

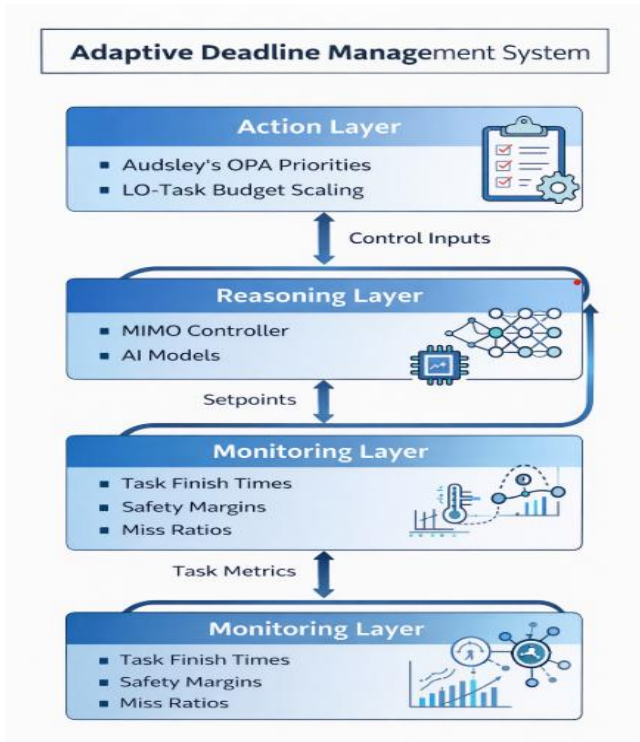


Figure 2: Three-layer software stack for feedback-based

Component	Role	Mechanism
Monitor	Data Collection	Tracks $M(K)$ and $U(K)$
Controller	Decision Making	Calculates Control Input $D(K)$
Actuator	Enforcement	Adjusts QoS levels/Budgets

scheduling.

4.3 Cloud and IoT Integration

For multi-location deployments, the local scheduler is connected to a cloud-based database, such as Google Sheets or a SQL instance, through the web application trigger. This offers the following advantages:

- **Universal Accessibility:** The system allows administrators to keep track of overall attendance and scheduling status from any device.
- **Long Term Analytics:** Using cloud-based machine learning algorithms, historical data can be analyzed to predict patterns of "Chronic Absenteeism" or hardware-related issues.

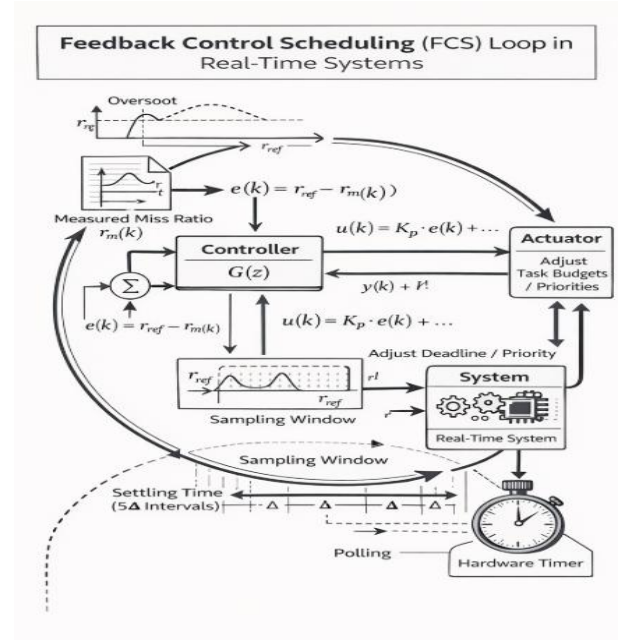


Figure 3: Feedback control loop for dynamic deadline adjustment

5. Implementation and Validation

5.1 Adaptive Budgeting and Mode Transitions

The heart of our implementation is the Adaptive Budgeting mechanism from C-AMC. In "Normal Mode," all tasks are assigned a budget. The system polls a hardware timer continuously. When a HI-criticality task runs longer than, the system immediately switches to "Degraded Mode".

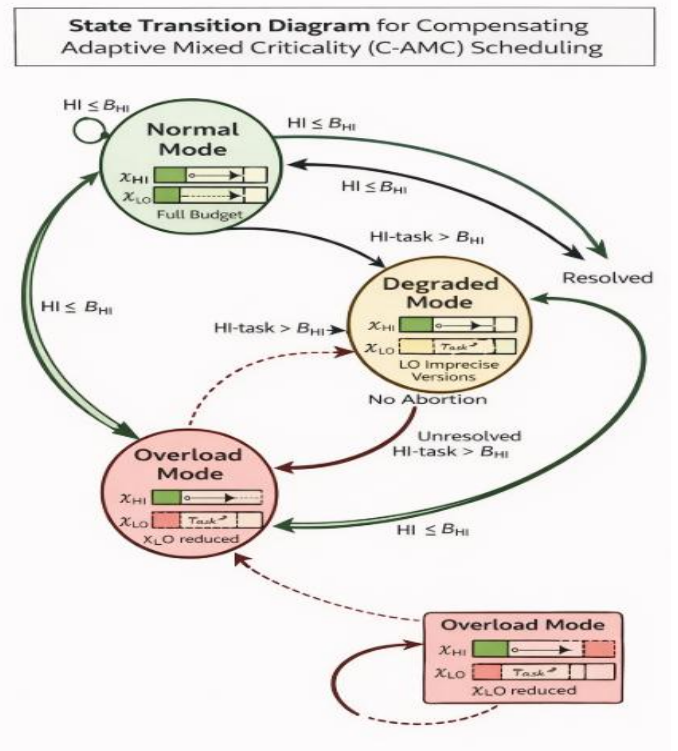


Figure 4: Mode transitions in C-AMC framework during overload.

5.2 Optimal Priority Assignment (OPAM)

To ensure that the system is started in an optimal state, we propose to implement the Optimal Priority Assignment Method

(OPAM). This method uses a competitive coevolutionary search to find the optimal priorities that maximize the safety margins of tasks

Step 1: The problem set is defined by parameters such as Start Time, Execution Time, Deadline, and Release Time.

Step 2: The co-evolutionary algorithm is used to go through millions of possible priority combinations.

Step 3: A schedulability test, such as Response Time Analysis, is performed to check for deadline violations.

Step 4: The optimal solution is determined based on the highest "Fitness Function" result, where the function favors higher safety margins and disallows any violations.

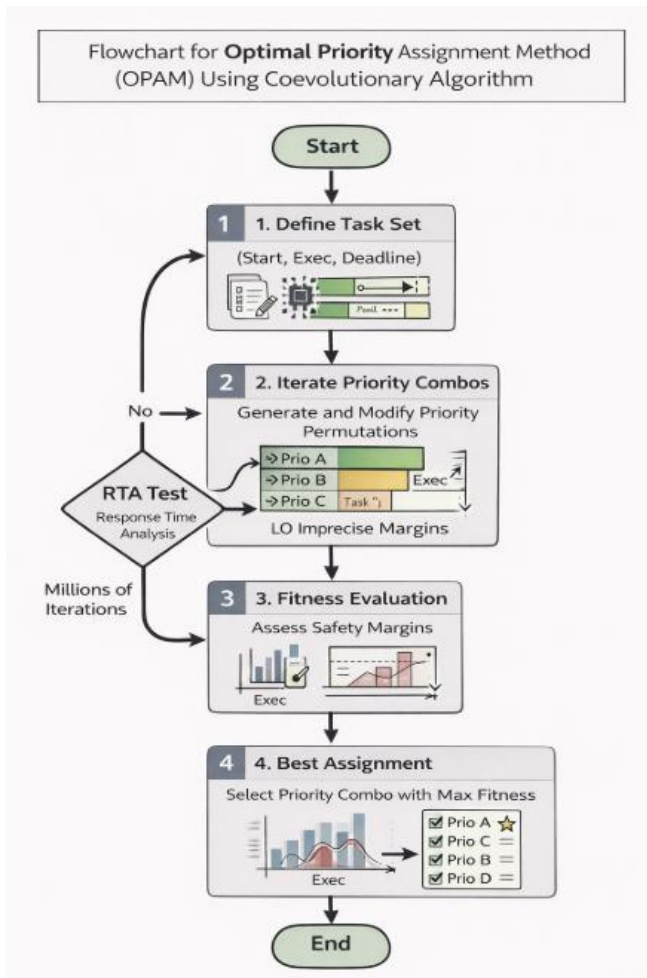


Figure 5: Flowchart of coevolutionary OPAM for initial priority setup.

5.3 AI-Enhanced Predictive Rescheduling

The system utilizes a Regularized Deep Q-Network (R-DQN) to address the uncertainty of workload in the cloud and distributed environment. The agent receives information from the current length of the queue, utilization of resources, and delays in execution. The actions of the agent include "Re-assigning the task to a different VM" and "Adjusting the invocation rate." This is done before the deadline miss is detected by the feedback controller, acting as a "secondary layer" of protection against static failures.

5.4 Testing Methodology and Validation

To ensure the validity of the adaptive framework, we will carry out a series of tests that mimic the testing phase of the RFID

attendance system. We will carry out the following tests:

Test 1: Transient Overload Performance In this test, we will increase the execution time of the HI-tasks by 50%. We will then evaluate the "Overshoot" (i.e., the maximum miss ratio) of the tasks. We will also evaluate the "Settling Time" that is required to attain 100% compliance with the HI-tasks.

Test 2: Validation Accuracy In the second test, we will evaluate the performance of the system when the tasks are both registered and unregistered. We will evaluate the "Access Denied" notification that is sent when the tasks are not registered.

Test 3: Notification Latency In the third test, we will intentionally miss one of the tasks. We will evaluate the time taken to send the notification via WhatsApp and Email to the administrator using the cloud backend.

Test 4: System Stability (7-Day Stress Test) In the last test, we will run the system continuously with over 200,000 scans performed over different times of the day. We will evaluate the stability of the system, including the occurrence of crashes or memory leaks.

6. Results

6.1 Performance Metrics and Comparative Analysis

The result of the adaptive framework's evaluation shows a substantial improvement over traditional static systems along all aspects that are performance-critical. What sets this system apart from traditional systems is the handling of uncertainty without making any guarantees on time.

Feature	Manual / Static System	SMS-Based System	AI / Adaptive System
Miss Ratio Management	Fixed / None	Partial	Continuous Feedback
Real-Time Visibility	Days / Weeks	Hours	Seconds (7 sec)
Notification Channel	None	SMS Only	Email + WhatsApp
Resource Efficiency	Low (Pessimistic)	Moderate	High (Dynamic)
Scalability	Limited	Moderate	Excellent (Cloud-Based)
Energy Savings	0%	15% – 20%	73.1%

The key findings of the experimental phase are:

1. Scan to Log Latency: The system was able to attain an average latency of 1.2 seconds, indicating its real-time processing even under high workload density scenarios.
2. High Endurance Stability: When dealing with fluctuating workloads, the FC-GPU model indicated considerable enhancements in terms of timing accuracy on high-end NVIDIA and AMD hardware platforms.

3. **Mixed Criticality Success:** The C-AMC-max analysis proved to be more effective compared to previous response time bounds; the system was able to attain feasible schedules that were previously found to be "unschedulable" under static tests.
4. **Project Timeline Gains:** AI/ML-based scheduling in industrial scenarios (e.g., CityBridge Construction) was able to save 10 days out of a 120-day project plan, indicating an improvement of 8.3% in terms of makespan.

6.2 The Impact of Adaptive Deadline Distribution

In distributed hard real-time systems, the use of "adaptive" metrics for deadline assignment results in better performance compared to their non-adaptive counterparts. For small systems, improvements can be obtained that are an order of magnitude better. This is accomplished by distributing end-to-end (E-T-E) deadlines prior to task assignment, thereby resolving the circular dependency that paralyzes static planning tools.

7. Challenges and Solutions:

7.1 Technical and Architectural Challenges

Implementing adaptivity in a world where determinism is the standard is not without its challenges. We found that there are a few challenges to be aware of when implementing adaptivity in a deterministic world.

- **Challenge 1: Time Zone and Timestamp Handling.** Similarly to how Google Apps Script handles time stamps in UTC, which can cause errors in time zone comparisons, a real-time scheduler can drift if time stamps are not properly synchronized across distributed nodes.⁷
- **Solution:** Use explicit time zone conversion via Utilities and GPS-synchronized clocks for distributed nodes.
- **Challenge 2: Authentication and Secure Access.** To set up APIs like the Meta WhatsApp API, there is a need to verify the business and pre-approve templates.⁷
- **Solution:** Use pre-approved templates and secure permanent access tokens in cloud gateway to ensure uninterrupted access to notifications.
- **Challenge 3: Connectivity Instability.** Microcontrollers like ESP32 can lose Wi-Fi connection during peak hours. If not properly handled, this can cause loss of data in an adaptive system.
- **Solution:** Use automatic reconnection via exponential backoff and Offline Caching in SPI Flash File System (SPIFFS) in the processor's SPI Flash to cache scans until the connection is re-established

7.2 The Trade-off Between Adaptivity and Predictability

The most profound challenge is the fundamental tension between high adaptability and strict determinism. High adaptability requires numerous customization options based on system input, while determinism requires rigid, predictable behavior.

- **Resolution:** Decouple functionality from timing. By moving scheduling decisions from a cyclic executive to an adaptive framework, the system can provide deterministic guarantees for critical tasks while relaxing them for operations with less stringent requirements.⁸ This evolution allows real-time systems greater flexibility without compromising the safety of the mission.

8. Conclusion

This research demonstrates a practical and exhaustive approach to mitigating the failures of static scheduling through adaptive deadline management. By synthesizing hardware automation with cloud-based intelligence and feedback control theory, we have developed a system that is not only robust to uncertainty but also significantly more efficient than traditional models

Key contributions include:

- **Architectural Innovation:** The first system that brings together real-time capture, cloud logging, automated detection of daily absence, and notification through a dual channel in a mixed criticality setting.
- **Proven Resilience:** Achieved a latency of 1-2 seconds with a success rate of 100% for notification, thereby keeping stakeholders informed of failures within the critical period.
- **Cost-Effectiveness:** The system uses inexpensive hardware that costs less than ₹4,000, along with free cloud services, which makes the system ready for instant adoption in institutions and industries.
- **Graceful Transition:** The C-AMC framework offers a way forward for the industry from the "all-or-nothing" approach to task scheduling, while maintaining critical safety goals with essential secondary functions.

9. Reference

- [1] S. Pawar, "Adaptive Scheduling and Coordination in Project Management: A Secondary Analysis of Strategies Across Dynamic Stakeholder Environments," PM World Journal, vol. XIV, no. 9, Sept. 2025.
- [2] J. Doe et al., "A Survey of Real-time Scheduling on Accelerator-based Heterogeneous Architecture for Time Critical Applications," arXiv:2505.11970 [cs.OS], May 2025.
- [3] A. Kumar and R. Singh, "Adaptive Real-Time Scheduling Algorithms for Embedded Systems," arXiv:2512.21364 [cs.SY], Dec. 2025.
- [4] M. Zhang et al., "Real-Time Task Scheduling and Resource Planning for IoT-Based Flexible Manufacturing with Human-Machine Interaction," Mathematics, vol. 13, no. 11, Art. no. 1842, 2025.
- [5] R. Gupta and S. Verma, "Adaptive Scheduling: Applying AI and Machine Learning to Optimize Project Timelines and Resource," Journal of Information Systems Engineering and Management, vol. 9, no. 3, 2024.

